Quarter-Car Suspension Modeling and Simulation

A project that builds and tunes a quarter-car suspension model in Simscape Multibody, evaluated with an automated road test suite. The model represents one vehicle corner (sprung body mass + unsprung wheel assembly, connected through suspension and tire compliance). A MATLAB test harness runs a suite of road profiles, computes objective comfort/packaging/road-holding metrics, and a parameter sweep tunes the suspension to meet constraints across all road cases.

Project files

- `model.slx` — the Simscape Multibody quarter-car model
- `initParams.m` — baseline parameters and default road input
- `roadSuite.m` — the 5 road test cases
- `scoreSuspension.m` — objective metrics and scoring
- `runAllTests.m` — automated test runner
- `tuneSuspension.m` — Ks/Cs parameter sweep
- `robustnessTest.m` — Monte Carlo robustness study
- `tools/` — shared helpers and the model-edit audit trail
- `results/` — generated tables and figures

P.S. To make all project files available, run the command below first:

```
addpath('tools');
```

Break Down the Problem

Problem statement:

A suspension has two main tuning parameters: spring stiffness `Ks` (N/m) and damping coefficient `Cs` (Ns/m). Changing them trades one goal against another. If we lower `Ks` — a softer spring — it reduces the acceleration felt by the vehicle body. Therefore, rides feels more comfortable. But the body then moves further with respect to the wheel, meaning the suspension needs more travel and can run out of clearance. Raising `Ks`, or raising `Cs` to add damping, limits that travel and helps keep the tire pressed against the road. However, this modification transmits more vibration to the occupants.

Because these goals conflict, there is no single best setting that can be found by intuition — the design has to be measured.

This project builds a quarter-car model of one vehicle corner and tests it against a fixed set of road disturbances. Each candidate `(Ks, Cs)` combination is scored on three criteria: ride comfort, suspension travel, and tire deflection. Changes of the design can be compared quantitatively before any hardware exists.

Requirements, constraints, and success criteria:

Functional requirements

- **The model** must represent one vehicle corner: a 350 kg body resting on a suspension spring and damper, which in turn sits on a 20 kg wheel supported by the tire's own stiffness

- **The model** must accept any road height profile as input, and output the three quantities the metrics need: body acceleration, suspension travel, and tire deflection
- **The test harness** must run every road case and score it automatically, with no manual steps

Constraints *(these numeric limits are our own engineering assumptions, not values given by the project brief)*

- Suspension travel ≤ 0.08 m — the suspension must not run out of clearance and slam into its end stops
- Tire deflection ≤ 0.02 m — the tire must stay pressed against the road. If it deflects too far it lifts off, and the vehicle loses braking and steering entirely

Note: the tire deflection limit turns out to sit very close to a hard physical threshold. This becomes important in the Interpretation of Results.

Criteria of Success

- Every one of the 5 road cases stays within both limits
- Among the designs that do, the one with the lowest average body acceleration is selected
- That design is then re-tested with `Ks` and `Cs` varied by ±10%, to check how sensitive it is to real component tolerances

Proposed solution:

We adopted a model-based workflow to minimize the time used for trial and error of tuning parameters. Steps:

1. Build the model with `Ks` and `Cs` as workspace variables, so they can be changed from a script instead of by editing the model by hand
2. Generate five road profiles in MATLAB: a bump, a pothole, rough road, washboard, and two bumps in succession
3. Define the metrics — body acceleration for comfort, suspension travel, and tire deflection — and combine them into a single score for ranking designs
4. Automate the suite so any design can be tested with one command
5. Sweep `Ks` and `Cs`, score every combination on all five roads, and pick the best design that stays within the limits
6. Re-test that design with ±10% variation to see how sensitive it is

Steps taken:

The work followed the plan above, but two stages turned out differently than expected:

1. **Repairing the model came first and took the longest.** The baseline model would not run at all, so four separate faults had to be found and fixed before any testing could begin. Only then were the four required signals wired out.
2. The road suite and scoring function were written next, as planned.
3. **The metrics had to be redefined partway through.** Measuring travel and deflection as raw distances gave meaningless numbers, so both were changed to measure movement away from the resting position instead.

4. The sweep of 90 designs across 5 roads (450 simulations) was run in parallel using `parsim`, taking about 5 minutes.
5. The robustness test then exposed a problem the sweep alone had hidden: the winning design sat right on a limit with no margin. This sent us back to re-examine whether the limit itself had been chosen sensibly.

Challenges faced:

The initial model did not run at all. During a discussion session, we uncovered four separate problems that could only be found in a few stimulation attempts: two separate "World" reference blocks, with all three joints attached to the one *not* connected to the solver; a road input joint not configured to let the solver work out the force needed to follow the commanded motion; body and wheel masses being calculated from shape and density instead of the values typed in, so the simulation ran with a 1000 kg body and a 3142 kg wheel instead of 350 kg and 20 kg; and springs set to be relaxed at zero length while the parts they connect sit 0.5 m apart, leaving both springs heavily compressed before gravity was even applied. The same mass fault later turned up in another version we tried to build from ground-up: the dialog still displays the intended mass even when that value is being ignored, so nothing looks faulty until the results are checked against hand calculations.

Defining the metrics correctly was less obvious than expected. Measuring suspension travel as the distance between body and wheel gave meaningless results at first, because the car's own weight already compresses the suspension by about 0.19 m before any road input arrives. That constant offset swamped the actual road-induced movement, so the metrics had to be redefined as the *change* from the resting position rather than the absolute distance.

The limit that mattered turned out to be one we could not control. After tuning, tire deflection was only just inside its limit, and no combination of `Ks` and `Cs` anywhere in the swept range could improve it meaningfully. Tire deflection is driven by how the wheel bounces on the tire, which depends on tire stiffness and wheel mass — neither of which was being tuned — and the rough road profile happens to contain vibration at exactly the frequencies that excite that bouncing. Recognizing this meant stepping back from the sweep results instead of assuming a better answer existed somewhere in the range.

The best design turned out to be fragile. The sweep selected a design sitting at 99.8% of the tire deflection limit — technically passing, but with almost no margin. When `Ks` and `Cs` were varied by ±10% to represent manufacturing tolerance, only 45% of trials stayed within the limits. If you ask an optimiser for the best design that satisfies a limit, it will settle right *on* that limit, leaving no room for real-world variation.

Task 1: Build a baseline quarter-car multibody model

The model represents one vehicle corner with three vertical degrees of freedom:

- **Road plate** — mass-driven by a From Workspace block carrying the road displacement
- **Unsprung mass** (wheel assembly, 20 kg) — connected to the road plate through the tire spring/damper (Kt = 200000 N/m)
- **Sprung mass** (body corner, 350 kg) — connected to the wheel through the suspension spring/damper, parameterized as `Ks` and `Cs`

Each body is grounded to the World frame through its own prismatic joint, giving one vertical DOF each; the spring/damper elements apply forces between them.

Model defects found and fixed. The baseline model would not compile or run as originally built. Four defects were found only by actually simulating it — each fix is recorded with its rationale in `tools/configureModelIO_step1*.m`:

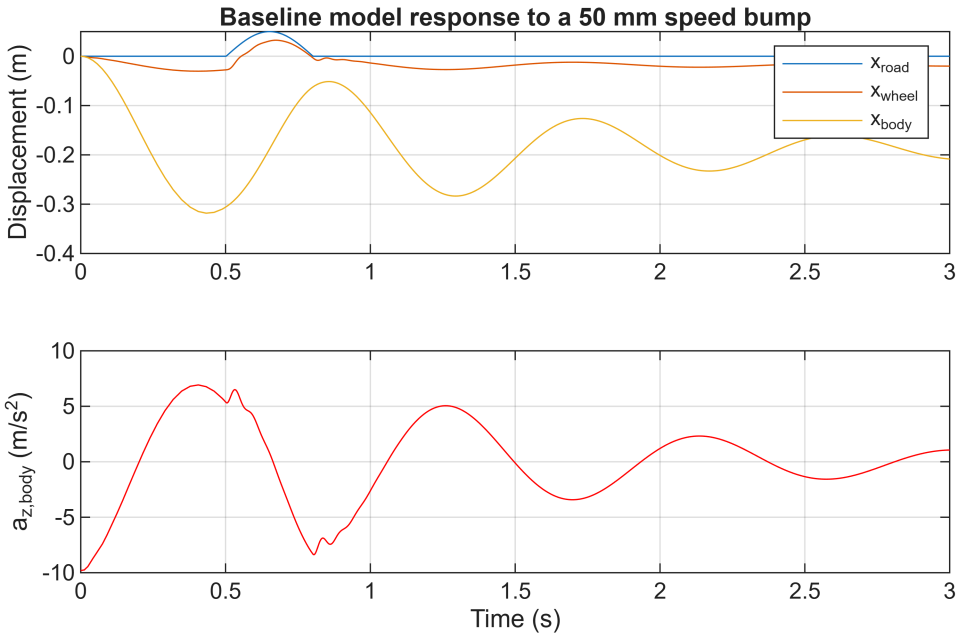
1. **Disconnected ground reference** — two separate World Frame blocks existed, and the one all three joints grounded to was not connected to the Solver Configuration.
2. **Road joint actuation mode** — the prescribed-motion joint required `TorqueActuationMode = ComputedTorque` to satisfy the solver's DOF/actuation balance.
3. **Wrong body and wheel masses** — both solids used `BasedOnType = 'Density'`, so Simscape computed mass from placeholder 1x1x1 m geometry (1000 kg body, ~3142 kg wheel) instead of the intended 350 kg / 20 kg.
4. **Spring preload mismatch** — both springs had `NaturalLength = 0` while the Rigid Transform blocks place the bodies 0.5 m apart at assembly, pre-compressing each spring by 0.5 m before gravity acted.

Verification. After these fixes the body settles at -0.190 m static sag, matching the hand-calculated series-spring result (mg/k) to within 0.03 mm, and initial body acceleration is exactly -9.80665 m/s^2 (free-fall before the spring engages).

```
initParams;   % sets Ks = 20000 N/m, Cs = 750 Ns/m, and a default road signal
open_system('model');
simOut = sim('model', 'ReturnWorkspaceOutputs', 'on');

x_road  = getLoggedSignal(simOut, 'x_road');
x_wheel = getLoggedSignal(simOut, 'x_wheel');
x_body  = getLoggedSignal(simOut, 'x_body');
az_body = getLoggedSignal(simOut, 'az_body');

figure;
subplot(2,1,1);
plot(x_road.Time, x_road.Data, x_wheel.Time, x_wheel.Data, x_body.Time,
x_body.Data);
legend('x_{road}','x_{wheel}','x_{body}'); ylabel('Displacement (m)'); grid on;
xlim([0 3]);
title('Baseline model response to a 50 mm speed bump');
subplot(2,1,2);
plot(az_body.Time, az_body.Data, 'r'); ylabel('a_{z,body} (m/s^2)'); xlabel('Time
(s)');
grid on; xlim([0 3]);
```




```
fprintf('Static body sag: %.4f m (hand-calculated: -0.1898 m)\n', x_body.Data(end));
```

```
Static body sag: -0.1898 m (hand-calculated: -0.1898 m)
```

Task 2: Create a road test suite (3-5 cases)

Five road profiles are generated in `roadSuite.m`, all sampled at 1 ms with a 1 s flat lead-in so every case starts from the same settled condition:

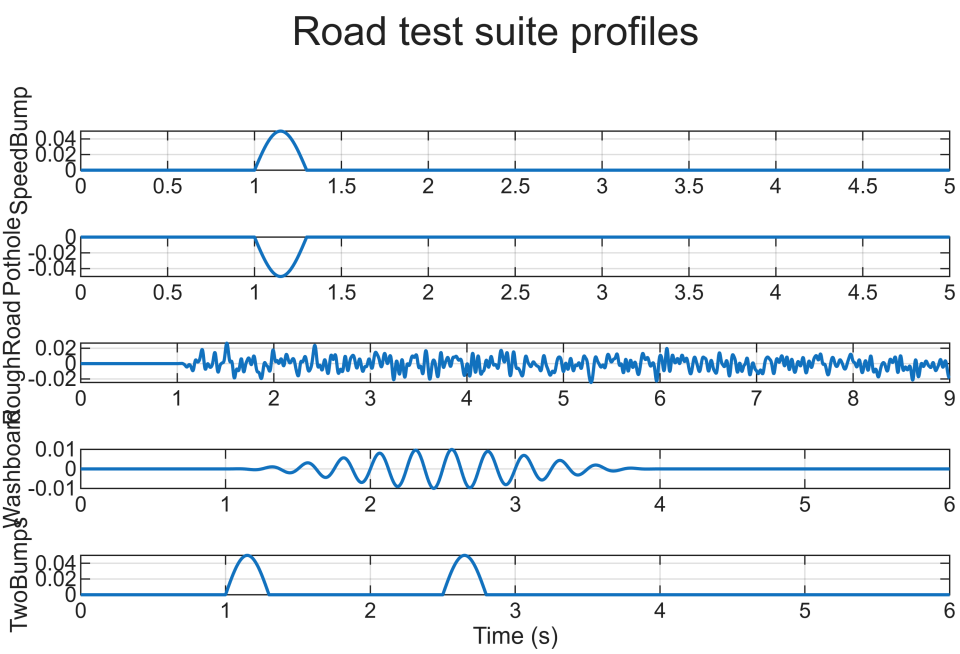
- **SpeedBump** — half-sine hump, 50 mm peak over 0.3 s
- **Pothole** — half-sine dip, -50 mm peak over 0.3 s
- **RoughRoad** — white noise low-pass filtered at 20 Hz, scaled to 8 mm RMS, with a 0.2 s raised-cosine fade-in to avoid a velocity discontinuity. Uses a fixed RNG seed (42) so every design is compared against the identical noise realization.
- **Washboard** — 4 Hz sinusoidal corrugation, 10 mm amplitude, 3 s burst with a smooth envelope
- **TwoBumps** — two 50 mm half-sine bumps 1.5 s apart, testing transient recovery

Each case also carries a `metricStart` time (1.0 s), used later to exclude the startup settling transient from the metrics.

```
cases = roadSuite();

figure;
for i = 1:numel(cases)
    subplot(numel(cases), 1, i);
    plot(cases(i).road.Time, cases(i).road.Data, 'LineWidth', 1.2);
    ylabel(cases(i).name); grid on;
end
```

```
xlabel('Time (s)');
sgtitle('Road test suite profiles');
```



Task 3: Log signals and define objective metrics

The model exposes four signals as root outports, wired from the joint sensing ports through PS-Simulink Converters: `x_road`, `x_wheel`, `x_body`, `az_body`.

`scoreSuspension.m` computes, per road case:

- **Comfort**: RMS and peak sprung-mass vertical acceleration
- **Packaging**: max suspension travel, `x_body - x_wheel`
- **Road holding**: max tire deflection, `x_wheel - x_road`

Important modeling choice. Suspension travel and tire deflection are measured as deviation from the static gravity-loaded ride height, not as absolute joint position. Without this, the constant -0.190 m gravity sag would dominate the packaging metric and obscure the actual road-induced motion. This matches how suspension travel is normally specified for a loaded vehicle.

Metrics are computed only after `metricStart` (1.0 s) so the startup settling transient is excluded. The function returns a struct of metrics, pass/fail flags against the constraints, and a single scalar score (RMS acceleration plus a penalty for any constraint violation).

```
r = scoreSuspension(simOut, 'SpeedBump');
disp(r);
```

```
      roadName: 'SpeedBump'
     rms_accel: 1.0746
    peak_accel: 5.0513
    max_travel: 0.1564
max_tireDefl: 0.0133
```

```
  travel_limit: 0.0800
tireDefl_limit: 0.0200
   pass_travel: 0
 pass_tireDefl: 1
          pass: 0
         score: 77.4527
```

Task 4: Build an automated test runner

`runAllTests.m` runs all five road cases at a given (Ks, Cs), scores each, prints a pass/fail line per case, returns a summary table, and saves a combined figure to `results/summary.png`.

Run at the **baseline** parameters (Ks = 20000 N/m, Cs = 750 Ns/m):

```
[baselineTable, baselineResults] = runAllTests(20000, 750);
```

```
Running 5 road cases at Ks=20000 N/m, Cs=750 Ns/m...
  [FAIL] SpeedBump   RMS accel=0.807 m/s^2  max travel=0.1016 m (lim 0.08)  max tire defl=0.0091 m (lim 0.02)  score
  [FAIL] Pothole     RMS accel=2.039 m/s^2  max travel=0.1584 m (lim 0.08)  max tire defl=0.0160 m (lim 0.02)  score
  [FAIL] RoughRoad   RMS accel=2.487 m/s^2  max travel=0.1281 m (lim 0.08)  max tire defl=0.0356 m (lim 0.02)  score
  [FAIL] Washboard   RMS accel=1.016 m/s^2  max travel=0.1074 m (lim 0.08)  max tire defl=0.0095 m (lim 0.02)  score
  [FAIL] TwoBumps    RMS accel=1.143 m/s^2  max travel=0.1016 m (lim 0.08)  max tire defl=0.0093 m (lim 0.02)  score

0 / 5 road cases passed constraints.
Saved results/summary.png
```

```
disp(baselineTable);
```

```
      roadName       rms_accel    peak_accel    max_travel    max_tireDefl    travel_limit    tireDefl_limit    pass
    _____________    _________    __________    __________    ____________    ____________    ______________    ____

    {'SpeedBump'}     0.8066        3.1109       0.10156       0.0090688          0.08             0.02         fals
    {'Pothole'  }     2.0395        6.9786       0.15841        0.016034          0.08             0.02         fals
    {'RoughRoad'}     2.4865        6.8031       0.12806        0.035564          0.08             0.02         fals
    {'Washboard'}     1.0159        3.2512       0.10739       0.0095123          0.08             0.02         fals
    {'TwoBumps' }     1.1434        3.8131       0.10156       0.0092757          0.08             0.02         fals
```

The baseline design fails every road case — suspension travel reaches 0.158 m on the pothole against the 0.08 m limit, and tire deflection reaches 0.0356 m on the rough road against the 0.02 m limit. This is what motivates the tuning step.

Task 5: Tune suspension parameters

Option A — parameter sweep. `tuneSuspension.m` sweeps Ks over 15000:5000:60000 N/m (10 values) and Cs over 500:250:2500 Ns/m (9 values), giving 90 candidate designs. Each design is simulated on all 5 road cases, so 450 simulations total, executed in parallel with `parsim` (with a serial `sim` loop as fallback).

For each design, the 5 per-case results are aggregated: comfort as the **mean** RMS acceleration across cases (overall ride quality), while travel and tire deflection use the **worst case** across cases (safety and packaging limits must hold in the worst condition). A design "passes" only if all 5 road cases satisfy both constraints.

The sweep takes roughly 5 minutes. Its results are cached, so this section loads them rather than re-running. To re-run the full sweep, uncomment the line below.

```
% [bestKs, bestCs, tuningTable] = tuneSuspension();   % ~5 min, regenerates the
cache

tuningTable = readtable('results/tuning_table.csv');
passing = sortrows(tuningTable(tuningTable.allCasesPass == 1, :), 'meanRMSAccel');
fprintf('%d of %d designs meet the constraints on every road case.\n', ...
    height(passing), height(tuningTable));
```

```
19 of 90 designs meet the constraints on every road case.
```

```
disp(head(passing, 8));
```

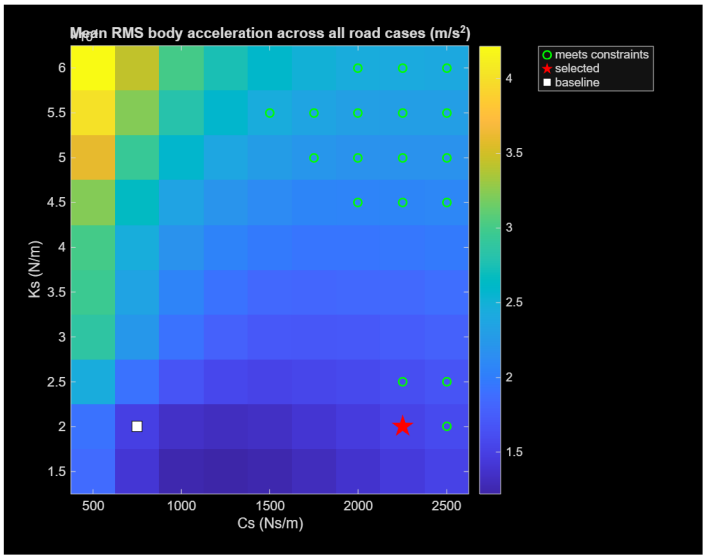
Ks	Cs	meanRMSAccel	meanScore	worstTravel	worstTireDefl	allCasesPass
20000	2250	1.5271	1.5271	0.05857	0.019954	1
20000	2500	1.5901	1.5901	0.052625	0.019849	1
25000	2250	1.6286	1.6286	0.05496	0.019562	1
25000	2500	1.6757	1.6757	0.05161	0.019533	1
45000	2000	2.0588	2.0588	0.064458	0.019984	1
45000	2250	2.0641	2.0641	0.059486	0.019532	1
45000	2500	2.0819	2.0819	0.055291	0.019587	1
50000	2250	2.183	2.183	0.054403	0.018606	1

```
bestKs = passing.Ks(1);
bestCs = passing.Cs(1);
fprintf('\nSelected design: Ks = %g N/m, Cs = %g Ns/m\n', bestKs, bestCs);
```

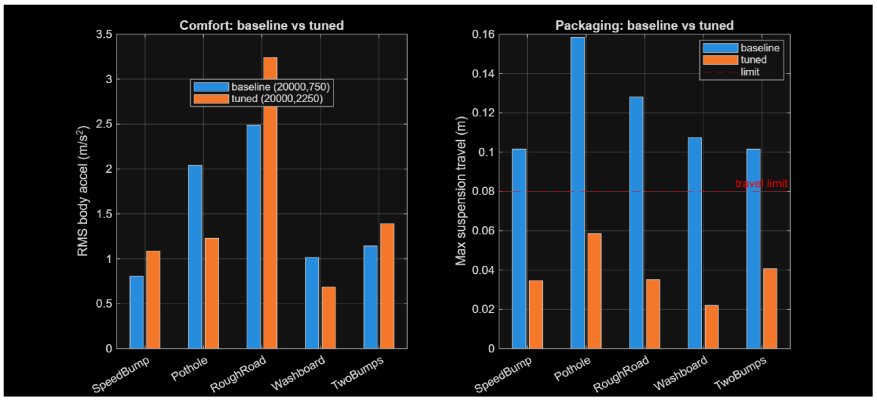
```
Selected design: Ks = 20000 N/m, Cs = 2250 Ns/m
```

Justification.

```
figure; imshow(imread('results/tuning_heatmap.png'));
```



```
figure; imshow(imread('results/tuning_comparison.png'));
```



The sweep selected **Ks = 20000 N/m, Cs = 2250 Ns/m** — the same spring rate as the baseline, with damping tripled. In damping-ratio terms this moves the sprung mass from zeta = 0.142 (badly under-damped) to zeta = 0.425 (well damped). The baseline spring rate was never the problem; the baseline was under-damped.

Comparing the two designs directly:

- Mean RMS body acceleration: 1.498 -> 1.527 m/s^2 (**1.9% worse**)
- Worst-case suspension travel: 0.158 -> 0.0586 m (**63% better**)
- Worst-case tire deflection: 0.0356 -> 0.0200 m (**44% better**)
- Road cases passing: 0/5 -> 5/5

The tuned design is therefore not more comfortable — it gives up 1.9% of ride comfort to cut suspension travel by nearly two thirds, which is what converts an infeasible design into a feasible one. Comfort was never the binding problem; excess travel was.

Task 6: Validate robustness with one simple variation

Component tolerance test. `robustnessTest.m` applies independent +/-10% uniform variation to Ks and Cs around the tuned design (20000, 2250) over 20 Monte Carlo trials, reruns the full 5-case road suite for each trial, and reports worst-case metrics and pass rate. A fixed RNG seed (7) makes the trial set reproducible.

As with the sweep, results are cached; uncomment to re-run.

```
% [robSummary, robTable] = robustnessTest(20000, 2250, 20);   % ~2 min

robTable = readtable('results/robustness_table.csv');
passRate = mean(robTable.allCasesPass);
```

```
fprintf('Pass rate: %d / %d trials = %.0f%%\n', ...
    sum(robTable.allCasesPass), height(robTable), 100*passRate);
```

```
Pass rate: 9 / 20 trials = 45%
```

```
fprintf('Worst-case RMS accel:       %.4f m/s^2\n', max(robTable.worstRMSAccel));
```

```
Worst-case RMS accel:       3.3100 m/s^2
```

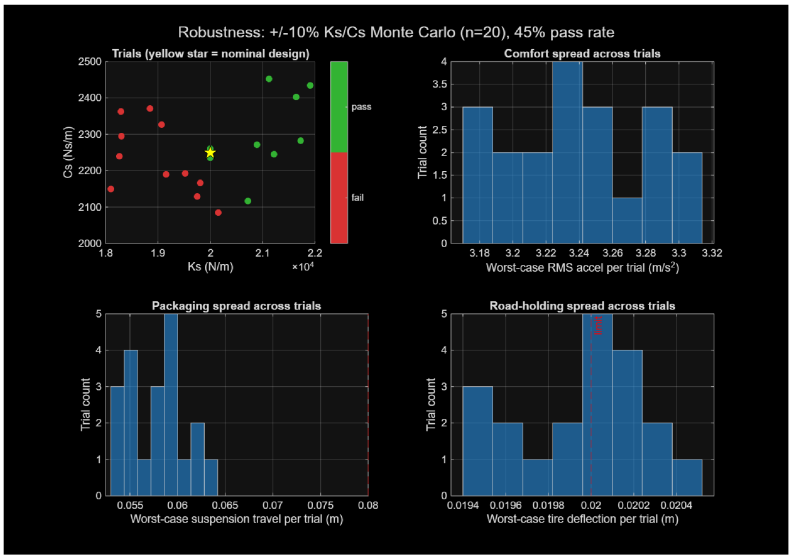
```
fprintf('Worst-case travel:          %.4f m (limit 0.08)\n',
max(robTable.worstTravel));
```

```
Worst-case travel:          0.0634 m (limit 0.08)
```

```
fprintf('Worst-case tire deflection: %.4f m (limit 0.02)\n',
max(robTable.worstTireDefl));
```

```
Worst-case tire deflection: 0.0205 m (limit 0.02)
```

```
figure; imshow(imread('results/robustness_summary.png'));
```



Only **45% of trials (9 of 20)** satisfy the constraints on every road case. The cause is explained in the next section: the selected design already sits at 99.8% of the tire-deflection limit, leaving essentially no margin for component variation.

Interpretation of Results

Physical and engineering meaning

- **The baseline design was under-damped, not too soft.** The sweep kept the original spring stiffness (`Ks` = 20000 N/m) and tripled the damping (`Cs` from 750 to 2250 Ns/m). The spring was never the problem — the suspension simply had too little damping to settle down after a disturbance.
- **The tuning traded almost no comfort for a large gain in travel.** Average body acceleration went from 1.498 to 1.527 m/s^2 (1.9% worse), worst suspension travel from 0.158 to 0.0586 m (63% better), worst tire deflection from 0.0356 to 0.0200 m (44% better), and road cases passing went from 0 of 5 to 5 of 5.
- **Comfort was never the binding problem — travel was.** The tuned design is not more comfortable. It gives up 1.9% of comfort to cut suspension travel by nearly two thirds, and that is what turns a failing design into a passing one.
- **The car has two very different bouncing motions.** The body bounces slowly on the suspension spring, about 1.2 times per second. The wheel bounces much faster on the tire, about 16.7 times per second. Suspension damping has strong control over the slow body motion — which is why travel improved so much — but very little control over the fast wheel motion, because that depends on the tire's stiffness and the wheel's mass instead.
- **Tire deflection is stuck against a hard physical limit.** Under the car's own weight the tire is already squashed by 18.1 mm. If the road deflects it much further than that, the force in the tire drops to zero and the wheel lifts off, losing braking and steering. Across all 90 designs, the best result achievable was 18.10 mm — 99.8% of that threshold. On the rough road, every single design leaves the tire on the verge of lifting off. The reason is that the rough road profile contains vibration close to the wheel's own fast bouncing rate.
- **This explains the poor robustness result.** Because the chosen design sits at 99.8% of the tire deflection limit, ordinary ±10% component variation is enough to push it over, and only 45% of the 20 trials stayed inside the limits. Asking for the best design that satisfies a limit naturally produces a design sitting exactly *on* that limit, with no margin left.

Limitations

- **The limits are our own assumptions, not stated requirements.** The 0.02 m tire deflection limit is slightly above the 18.1 mm point where the tire would actually lift off. Applying a stricter 0.018 m limit instead leaves **0 of 90 designs passing** — so the fact that any design passes at all depends on a somewhat generous assumption.
- **Only `Ks` and `Cs` were tuned**, yet the limit that actually mattered is controlled by the tire stiffness and wheel mass, which were fixed throughout.
- **The sweep is coarse** — `Ks` in steps of 5000 N/m and `Cs` in steps of 250 Ns/m, so a better setting could sit between the values tested.
- **The spring and damper are both perfectly linear** — no end stops, no stiffening at large travel, and no difference between compression and rebound, so behaviour at large movements is more optimistic than reality.
- **Only one corner of the car is modelled**, with each part moving only up and down — no pitching, rolling, or weight transfer between wheels.
- **Every run starts from an unloaded position** and settles under gravity first. Metrics ignore the first second to avoid this, but starting the model already at rest would be cleaner.
- **The robustness test varied only `Ks` and `Cs`** — mass and tire stiffness tolerances were not included, so the 45% pass rate is probably optimistic.

- **The rough road uses one fixed random profile**, so that result reflects a single example rather than an average over many rough roads.

Practical next steps

1. **Add tire stiffness as a third tuning variable.** Road holding is controlled by the wheel's fast bouncing motion, which `Ks` and `Cs` cannot reach. This is the single most useful extension.
2. **Re-tune with a required safety margin.** Instead of accepting any design that merely stays inside the limits, require it to stay within 90-95% of them. In this sweep only the 95% level produces any passing designs — 9 out of 90, and the best of those is 43% worse for comfort, which puts a concrete price on robustness.
3. **Re-examine how harsh the rough road is.** Its vibration sits right on the wheel's bouncing rate. Comparing it against a standard road roughness classification would show whether it is realistic or unfairly severe.
4. **Look at the results in the frequency domain.** Plotting how much road vibration reaches the body at each frequency would show both bouncing motions directly, instead of working them out by hand.
5. **Add end stops to the suspension.** Since travel was the original problem, modelling what happens when the suspension runs out of room would make large bumps far more realistic.
6. **Reduce the wheel mass.** A lighter wheel bounces less, which directly improves the tire deflection margin that suspension tuning could not reach.